

Understanding the Retrieval-Augmented Generation (RAG) Pipeline

Introduction

Retrieval-Augmented Generation, commonly known as RAG, is an AI architecture that combines information retrieval with a Large Language Model (LLM). Instead of relying only on the knowledge stored inside the language model, RAG allows the model to search external documents and use that information while generating answers.

Traditional language models answer questions based only on what they learned during training. If the requested information was not part of the training data, or if it has changed after the model was trained, the answer may be incorrect or incomplete.

RAG solves this problem by retrieving relevant information from an external knowledge base before asking the language model to generate the final response.

Because of this capability, RAG is widely used in chatbots, document assistants, enterprise search systems, customer support platforms, legal applications, healthcare systems, and research tools.

Components of a RAG System

A typical RAG system consists of several independent modules.

The first module is the document loader.

The second module is the chunking system.

The third module generates embeddings.

The fourth module stores embeddings inside a vector database.

The fifth module retrieves the most relevant chunks.

The final module sends those retrieved chunks to a language model to generate an answer.

Each module has a single responsibility, making the system easier to understand, maintain, and improve.

Document Loading

The document loader reads files from storage.

It supports formats such as PDF, DOCX, TXT, and Markdown.

Each document is converted into plain text.

After conversion, the remaining pipeline works only with text.

The original document format is no longer required during embedding generation.

If a PDF contains ten pages, the loader extracts the text from every page.

If a DOCX file contains paragraphs and headings, those are converted into plain text.

The output of the document loader is always one long string.

Chunking

Large documents cannot be embedded efficiently as a single block.

Instead, they are divided into smaller pieces called chunks.

Each chunk contains only a portion of the document.

Chunk size is usually measured in characters or tokens.

Our project currently uses character-based chunking.

A chunk overlap is also maintained.

Overlap means that a small portion of one chunk is repeated in the next chunk.

This prevents important information from being lost at chunk boundaries.

Without overlap, a sentence split between two chunks could become difficult to understand.

Chunking improves retrieval quality.

It also reduces memory usage.

Embeddings

Embeddings are numerical representations of text.

Instead of storing words directly, the embedding model converts text into vectors.

Our project uses the `nomic-embed-text` model running locally through Ollama.

Each chunk is converted into a vector with hundreds of numerical values.

In our project, each embedding contains 768 dimensions.

Similar meanings produce similar vectors.

Even if two sentences use different words, their vectors may be close together if they have similar meaning.

This enables semantic search instead of keyword matching.

ChromaDB

ChromaDB is the vector database used in this project.

It stores the document chunk, its embedding, and its metadata.

Metadata includes the original filename, file type, and chunk number.

Each stored chunk has a unique identifier.

Our collection is named `documents`.

A collection is similar to a table in a relational database.

Each chunk is stored as a separate record.

Large documents produce many records.

Small documents may produce only one record.

Ingestion Process

The ingestion pipeline automates document processing.

It scans the data folder.

Each supported file is loaded.

The text is extracted.

The text is split into chunks.

An embedding is generated for every chunk.

Each chunk and its metadata are stored in ChromaDB.

This process continues until every document has been processed.

If duplicate chunk IDs are detected, they are skipped.

Query Process

When a user asks a question, the question is converted into an embedding.

That embedding is compared with every stored embedding in ChromaDB.

The database returns the most similar chunks.

The number of returned chunks is controlled by the Top-K parameter.

Our current configuration uses Top-K equal to three.

These retrieved chunks become the context for the language model.

The original documents are not sent directly.

Only the most relevant chunks are included.

This makes the response faster and more focused.

Response Generation

The retrieved chunks are combined with the user's question.

Both are sent to the local Phi-3 language model running through Ollama.

The language model reads the supplied context.

It generates an answer based only on that retrieved information.

This greatly reduces hallucinations.

If the required information is not present in the database, the model should indicate that it cannot answer confidently.

Advantages of RAG

RAG allows language models to answer questions using private documents.

The knowledge base can be updated without retraining the model.

New PDFs can simply be added to the data folder.

After running the ingestion pipeline again, the new information becomes searchable.

This makes maintenance simple.

RAG also keeps sensitive documents local when using local models such as Ollama.

No internet connection is required after the models are downloaded.

Our Local Project

Our implementation is written entirely in Python.

The project uses Ollama to run both the embedding model and the language model locally.

ChromaDB provides persistent vector storage.

The ingestion script processes all supported documents inside the data directory.

The query script will later retrieve the most relevant chunks.

The retrieved context will then be sent to Phi-3.

The final answer will be generated completely offline.

This project demonstrates the complete lifecycle of a local Retrieval-Augmented Generation system.

Understanding each module individually makes the complete pipeline easier to understand.

By combining document loading, chunking, embeddings, vector search, retrieval, and language generation, we create a powerful system capable of answering questions from our own documents while keeping all processing on the local computer.